# sootopolis (3% of your grade)

CS1010 PE3 - AY 25/26 Sem 1

## 1 Background and Task

If you play the old Pokemon games, you might have seen Sootopolis gym. In that gym, you as the player have to navigate a rectangular grid of $r \times c$ ice tiles (moving in only either Up/Down/Left-/Right directions), with $r$ rows, and $c$ columns. Once you are on an ice tile, the ice will crack, so **you can only step on each tile once**. If you re-visit a cracked tile, it will break and your character falls though the floor. The grid also has a few **rocks**. If the player were to try to walk into a rock, their position does not change. (e.g., like trying to walk into a wall.)

You'll be given: the number of rows and columns, an array of points which represent the positions of the rocks, and a sequence of movements in the form of a **null-terminated string**, with characters "U", "D", "L", "R".

**Movement rules:**

1. You will start on the top left corner (as row 0, column 0).
2. The character 'U' represents reducing your character's row by 1.
3. The character 'D' represents increasing your character's row by 1.
4. The character 'L' represents reducing your character's col by 1.
5. The character 'R' represents increasing your character's col by 1.
6. Walking into a rock will leave your position unchanged.
7. Walking off the grid ends the walk. (Simulate no further steps.)
8. Any tile that is stepped on twice (including the starting tile) will crack and ends the walk. (Simulate no further steps.)

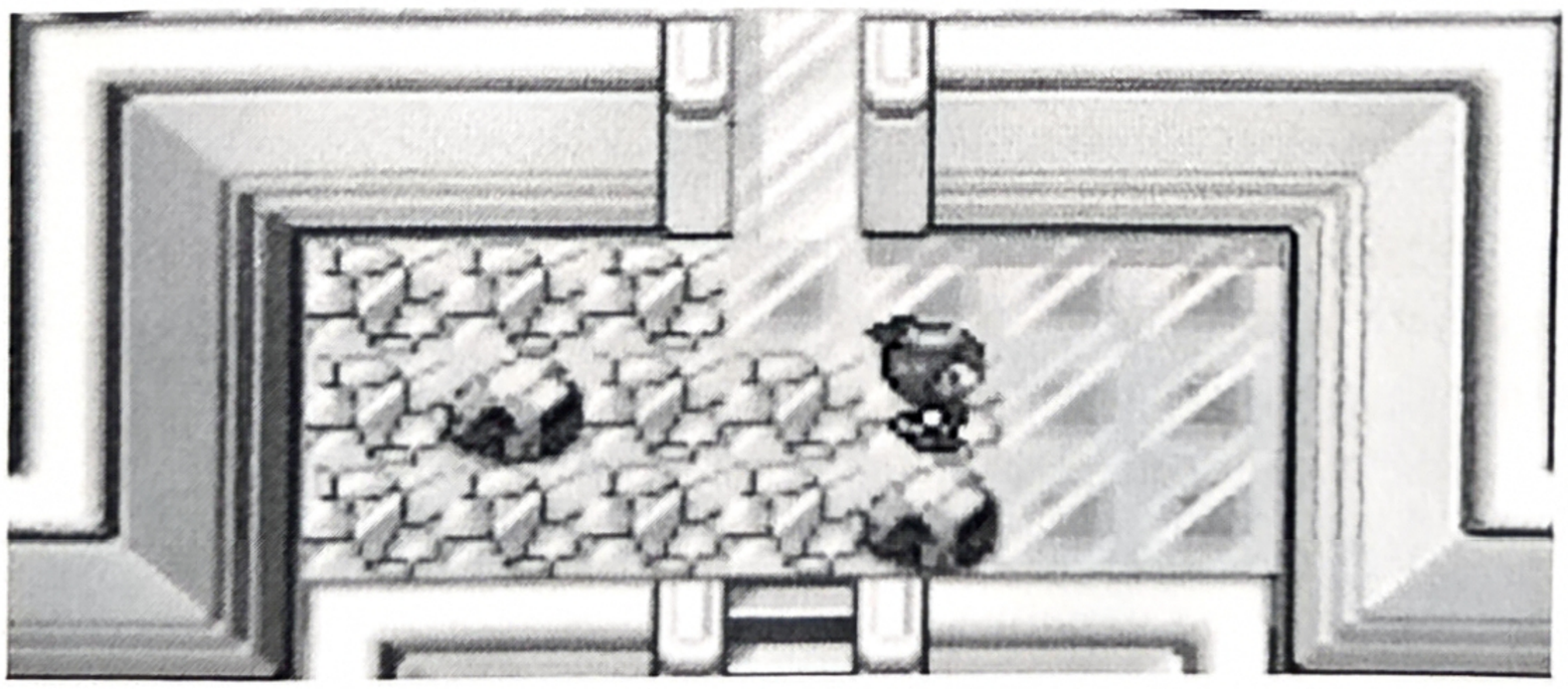

Figure 1: Example where there are 2 rocks, and the tiles to the left of (and below) the player have been stepped on, and therefore cracked and cannot be stepped on again. Tiles on the right are uncracked and can still be stepped on at least once by the player.

**Your Task:** Simulate moving through the grid of ice tiles based on the rules above. We will allocate the state and print out the state for you. Your job is to **(a) update the player's position, (b) mark on our grid, whether a tile has been stepped on or not**, and **(c) indicate if the player fell**. We will print out the required symbols for you.

## 2 Code

You are given the following partially filled function in `student.c` which you should implement. Only make changes within `student.c`. **We recommend not to edit the already-provided code.**

```
void sootopolis(size_t row, size_t col, Point *rock_list,
                size_t num_rocks, char *movement_str){
    Point curr_position = {0, 0};
    bool **ice_grid = calloc(row, sizeof(bool *));
    bool fell = false;
    for(size_t r = 0; r < row; ++r){
        ice_grid[r] = calloc(col, sizeof(bool));
    }
    ...
    // our remaining code to print and de-allocate
}
```

The Point struct is the following:

```
typedef struct Point {
    size_t row;
    size_t col;
} Point;
```

You are also given **two helper functions:**

1. `bool in_rock_list(Point point, Point *rock_list, size_t num_rocks)`, a function which will return true if `point` is a rock, and false otherwise.
2. `bool within_grid(Point position, size_t row, size_t col)`, a function which returns true if a position is within the ice grid, and false otherwise.

You may use these if you find it useful.

`ice_grid` is a boolean grid that will track whether a tile has been stepped on before or not. You should track a player's steps using this grid. `fell` is also a boolean that will track whether the player has fallen through the grid or not (due to revisiting the same tile twice). `curr_position` is a Point indicating the player's current location.

**Your job is to update three variables:** `curr_position`, `ice_grid` **and** `fell` based on the movements being made.

Implement all your code in `student.c`. To compile, run `make`, to test, run `make test`, and to manually run, do `./sootopolis`. Input format given based on examples in the next section.

**Note:** We will handle all input reading, and printing (and formatting for printing) for you. We have also handled freeing any heap allocations that we have made. Just implement the simulation.

# 3 Sample Input

The **input format** is given as follows:

1. The first line specifies the row and the column of the grid
2. Followed by 0 or more lines of row and column values (on the same line) specifying rock positions, if any. (There should never be a rock on position 0, 0.)
3. Followed by a ';' character on its own line
4. Lastly, a single string line of 'R', 'L', 'D', 'U', specifying the sequence of movements the character makes. It may walk off the grid, it may walk into a rock and continue walking.

The **output printing** (handled by us) will draw the grid, where:

1. Unstepped-on ice-tiles will have the '~' (tilde) character.
2. Stepped on ice-tiles will have the '*' character.
3. Rock tiles will have the 'R' character.
4. If the character is on the grid and **hasn't fallen down**, it has the '@' character.
5. If the character is on the grid and **has fallen down**, it has the 'O' character.

**Sample Input 1:**

```
5,5
;
DDRRRDDLL
```

Expected Program Output 1:

```
*~~~~
*~~~~
****~
~~~*~
~@**~
```

---

Sample Input 2:

```
5,5
2,2
;
DDRRRDDLL
```

Expected Program Output 2:

```
*~~~~
*~~~~
**R~~
~*~~~
**~~~
```

---

Sample Input 3:

```
5,5
;
DDRRRDDLLUUUUUUUUUUUU
```

Expected Program Output 3:

```
*~~~~
*~~~~
*0**~
~*~*~
~***~
```

---

Sample Input 4:

```
5,5
1,0
;
DDRRRDDLLL
```

Expected Program Output 4:

```
****~
R~~*~
@***~
~~~~~
~~~~~
```

---

Sample Input 5:

```
5,5
1,0
0,1
;
RRDDRRDD
```

Expected Program Output 5:

```
@R~~~
R~~~~
~~~~~
~~~~~
~~~~~
```

# changedir (4% of your grade)

CS1010 PE3 - AY 25/26 Sem 1

## 1 Background and Task

You'll be given two strings that represent two directories. For simplicity, a directory comes in the form of a bunch of lowercase alphabetical letters, and are separated by '/' characters. A directory will not start with a '/' letter.

e.g., a directory like `a/b/c` means we're in a folder `c` which is in a folder `b` which is in a folder `a`.

Let's say we wish to change our directory from this directory to something else. We need to specify a sequence of names, also separated by '/' characters. The name '..' is special, because it means we will go up 1 directory.

**For example**, if we start at directory `a/b/c`, and we run the command `cd ../../newfolder`, we will firstly go up 2 directories, landing in folder `a`. We will then go into a new subfolder of `a`, called `newfolder`. Therefore, our final directory is `a/newfolder`.

**Your Task:** Given a start and an end directory, output the **shortest possible** `cd` **command** that gets you from the start directory to the end directory.

**Note:** We promise to always give 2 distinct directories, and the input format is always valid. Our directories never start with '/' and never end with '/'.

## 2 Code

You are given the following function(s) in `student.c` which you should implement. Only make changes within `student.c`. **You can modify any of the code provided if you want.**

```
void changedir(char *dir_from, char *dir_to){
    char *change_cmd = NULL;
    // add your code here (you can add other functions too)
    printf("cd %s\n", change_cmd);
    free(change_cmd);
}
```

We have declared a variable `change_cmd` for you to use. We have also prepared a function for you: `void append_to_str(char **str_ptr, char *to_append)`

`append_to_str` takes **a pointer to a string** (`str_ptr`), and another **string** (`to_append`). If `str_ptr` **points to a null-pointer**, `append_to_str` will **allocate** a string and write into it the contents of `to_append`. Otherwise, `append_to_str` will resize the string pointed at by `str_ptr` so that it is large enough to fit both `*str_ptr` and `to_append`. It then appends `to_append` to

the string pointed at by `str_ptr`. You may choose to use this function if you find it useful. (Think of this like `strcat` from the standard library except we handled resizing for you.)

Implement all your code in `student.c`.
To compile, run `make`, to test, run `make test`, and to manually run, do `./changedir`.
**Note:** We will handle all input reading, and printing (and formatting for printing) for you. We also gave you `free(change_cmd)` at the end of the function. If you make any other allocations, you'll need to free them for yourself. Just implement the simulation.

# 3 Sample Input

The input format is just 2 lines which specify the directories. **Sample Input 1:**

```
a/b/c
a/b/c/d
```

**Expected Program Output 1:**

```
cd d
```

---

**Sample Input 2:**

```
a/b/c
a/b/d
```

**Expected Program Output 2:**

```
cd ../d
```

---

**Sample Input 3:**

```
a/b/c
a/b
```

**Expected Program Output 3:**

```
cd ../
```

---

**Sample Input 4:**

```
a/b
a/b/c
```

**Expected Program Output 4:**

```
cd c
```

# 4 Hint from Eldon

The way I solved this was to think of 3 possible cases.

1. When the source directory is a prefix of the target directory.
2. When the target directory is a prefix of the source directory.
3. When neither are a prefix of each other.

In the examples section I've already laid out the 3 cases, if you've noticed. Think about how to handle the 3 cases separately.

# parse (4% of your grade)

CS1010 PE3 - AY 25/26 Sem 1

## 1 Background and Task

**Your Task:** Read from standard input a single line that represents a C function declaration. You need to parse this and populate our struct with **heap allocated, null-terminated** strings such that you parse **these four properties of the function**:

1. Return type.
2. Function name.
3. List of parameter types, if any. Otherwise the list is set to `NULL`.
4. Number of parameters.

For example, on input `void foo(int)`, you need to populate the struct with these four **heap allocated, null-terminated** strings:

1. Return type = "void".
2. Function name = "foo".
3. List of parameter types = {"int"}.
4. Number of parameters = 1.

**Note:** We promise that:

1. Function names are alphanumeric and have no spaces in them.
2. Anything to the left of the function name is considered the function return type.
3. Parameter types if any, are separated by commas. And we will not add leading or trailing spaces between parameter types. Don't change the types (e.g., changing char * to char*).
4. **You will not get function pointers as parameters, so you can be 100% sure that commas will separate your parameter types.**
5. We promise to always give valid declarations. So you don't have to handle invalid inputs.

**Hint:** If you're having trouble figuring out how to tell what is a function name or not. Function names are alphanumeric are immediately followed by a '(' character.

## 2 Code

You are given the following function(s) in `student.c` which you should implement. Only make changes within `student.c`. **You can modify any of the code provided if you want.**

```
char *get_name(char *str){
}

char *get_return_type(char *str){
}
```

```
char **get_parameter_types(char *str, size_t *num_params_ptr){
}

Declaration parse_func_decl(){
    Declaration decl = {"\0", "\0", NULL, 0};
    char *line = NULL;

    // Read input here: into the variable: line

    char *name = get_name(line);
    char *ret = get_return_type(line);

    size_t num_params = 0;
    char **parameter_types = get_parameter_types(line, &num_params);

    decl.name = name;
    decl.return_type = ret;
    decl.parameter_types = parameter_types;
    decl.num_types = num_params;

    // Note: Cleanup (if needed) the variable: line
}
```

The relevant struct is the following:

```
typedef struct Declaration {
    char *return_type;
    char *name;
    char **parameter_types;
    size_t num_types;
} Declaration;
```

In `parse_func_decl()`, you'll need to read from the standard input. You should implement the respective code for `get_name`, `get_return_type`, `get_parameter_types`. They should all return **heap allocated, null-terminated** strings. Furthermore, `get_parameter_types` needs to set `num_params` to the number of parameters.

If there are **no parameters**, set `parameter_types` to `NULL`. For example the declaration `int foo()` has no parameters. If there **are parameters** (even void will count), then `parameter_types` needs to be an array of strings.

To compile, run `make`, to test, run `make test`, and to manually run, do `./func-parse`. Input a

single line that represents a function declaration. **You need not include the semi-colon.**

**Note:** We will handle all printing (and formatting) for you. You have to make the relevant heap-allocations, and handle input reading. We will free them for you anything put into `Declaration`.

# 3 Sample Input

**Sample Input 1:**

```
void foo()
```

**Expected Program Output 1:**

```
Return type: [void]
Name : [foo]
No inputs taken since num_types set to 0.
```

---

**Sample Input 2:**

```
int main(void)
```

**Expected Program Output 2:**

```
Return type: [int]
Name : [main]
        Parameter 0 : [void]
```

---

**Sample Input 3:**

```
void *foo()
```

**Expected Program Output 3:**

```
Return type: [void *]
Name : [foo]
No inputs taken since num_types set to 0.
```

---

**Sample Input 4:**

```
void foo(int,char *)
```

**Expected Program Output 4:**

```
Return type: [void]
Name : [foo]
        Parameter 0 : [int]
        Parameter 1 : [char *]
```

# directory (4% of your grade)

CS1010 PE3 - AY 25/26 Sem 1

## 1 Background and Task

In this task we're going to implement a popular linux tool called `tree` (let's call ours `dirtree`). It's a tool that basically prints out your directory (with all folders and files) in the shape of a tree.

For example, if let's say in our current directory, we had two folders, `a` and `b`. And let's say in folder `a` we had files `Makefile` and `main.c`. Whereas in folder `b`, we had files `student.c` and `student.h`.

Then the `tree` program would show this:

```
.
├── a
│   ├── Makefile
│   └── main.c
└── b
    ├── student.c
    └── student.h
```

**Explanation:** Since `a` and `b` are in our current directory, they are shown on the **4th column** of the output. Since we are printing `a` first, this means we'll see the symbol "`├──` " before `a`. And since `b` is the last in our current directory, we'll see "`└──` " before `b`.

Likewise, since `Makefile` and `main.c` are inside the folder `a`, the same explanation applies for why we see each of the symbols behind them.

Furthermore, since when printing out the contents of `a`, we still have to print out the contents of `b` later on, that's why for example the line for `Makefile` is basically:

```
│   ├── Makefile
```

To understand this, think of it this way, **the first 4 characters** on this line are "`│   `". This is because the tree branch needs to extend to folder `b` later on. **The next 4 characters** are then "`├──` ". Hence you should think of the line as being done up like so:

1. Firstly we print 4 characters "`│   `"
2. Next we print 4 characters "`├──` " (because `Makefile` is **not** the last item)
3. Lastly, we print the name of the item itself, in this case: `Makefile`

On the other hand, something like `main.c` has the following:

```
│   └── main.c
```

Similar logic as before, except `main.c` is the last item in the `a` folder.

1. Firstly we print 4 characters "│ "
2. Next we print 4 characters "└── " (because `main.c` is the last item)
3. Lastly, we print the name of the item itself, in this case: `main.c`

More examples in the examples section.

**Your Task:** Given a struct `Dir` that describes the directory that we are in. Print out the the directory as a tree. We will not distinguish between files and folders so don't worry about that. The directory items should be printed out **in the order in which they appear in the list** (no need to sort anything). More details in the code section when we explain the definition of `Dir`.

**Note:** We promise that:

1. We will always provide at least one folder/file.
2. The directories are specified as a list of names separated by slashes '/'.
3. Directories never start with a '/'.
4. Directories never end with a '/'.
5. Folder names are at most 32 chars.
6. Directories will have at most 999 slashes '/'. (I.e. the deepest we go is 1000 folders.)

# 2 Code

You are given the following empty function(s) in `student.c` which you should implement. Only make changes within `student.c`.

```
void dirtree(Dir *root, size_t depth, bool inbetween_depths[1000]){

}
```

The struct `Dir` is given as:

```
typedef struct Dir {
    char name[32];
    struct Dir **subdirs;
    size_t num_subdirs;
} Dir;
```

The current directory's name is specified in `name`. The list of subdirectories is given as **an array of pointers to `Dir`**. The number of subdirectories is specified in `num_subdirs`.

For the other two arguments: we will initially call `dirtree` with `depth` set to 0, and `inbetween_depths` as an array of 1000 bools, all initially set to false. You're not obliged to do anything with these, just that they might be helpful.

We also give you **five helper functions** for printing (see Section 4, after the examples).

## 2.1 Example

For example, given the previous layout, where we had the following directories:

1. `a/Makefile`
2. `a/main.c`
3. `b/student.c`
4. `b/student.h`

In our starting `Dir *root`, it is **always** that `root->name` is ".".

Also, we will have that:

1. `root->subdirs[0]` points to a directory that represents `a`
2. `root->subdirs[1]` points to a directory that represents `b`
3. `root->num_subdirs` is set to 2.

This also means, for example:

1. `root->subdirs[0]->subdirs[0]` points to a directory that represents `Makefile`
2. `root->subdirs[0]->subdirs[1]` points to a directory that represents `main.c`
3. `root->subdirs[1]->subdirs[0]` points to a directory that represents `student.c`
4. `root->subdirs[1]->subdirs[1]` points to a directory that represents `student.h`

**You should print out the names in the following order:**

1. Name of the current directory first
2. From index `0` until `num_subdirs - 1`, print out what the subdirectories are.

**Note:** We will handle input reading and parsing for you. You should handle the output (you may use our helper function).

# 3 Sample Input

**The input format:** Just supply on each line a directory and to end your input, on the very final line, write ';'. We will then sort it for you (Eldon has called `qsort` for you don't worry, you dont need to do anything) and create the `dir` struct for you. All you have to do is print out based on the format we have described.

**Sample Input 1:**

```
a
;
```

**Expected Program Output 1:**

```
.
└── a
```

**Sample Input 2:**

```
a
b
c
;
```

**Expected Program Output 2:**

```
.
├── a
├── b
└── c
```

---

**Sample Input 3:**

```
a/1
a/2
;
```

**Expected Program Output 3:**

```
.
└── a
    ├── 1
    └── 2
```

---

**Sample Input 4:**

```
a/1/a
a/1/b
a/2/a
a/2/b
;
```

**Expected Program Output 4:**

```
.
└── a
    ├── 1
    │   ├── a
    │   └── b
    └── 2
        ├── a
        └── b
```

---

**Sample Input 5:**

```
usr/bin/bash
etc/slurm
var/log
;
```

**Expected Program Output 5:**

```
.
├── etc
│   └── slurm
├── usr
│   └── bin
│       └── bash
└── var
    └── log
```

## 4 Helper functions

There are 4 functions that will help print branches:

1. `void print_middle_branch()`
2. `void print_ending_branch()`
3. `void print_4_spaces()`
4. `void print_vertical_branch()`

Respectively, these print out:

1. ├──
2. └──
3. 
4. │

And lastly a function called:

`print_branches(size_t depth, bool inbetween_depths[1000])`. If say `inbetween_depths` is set to something like `true, false, false, true`, and set `depth = 4`, it will (from left to right) print out:

1. │ , then
2.  , then
3.  , then
4. │ (it does not end on a newline in case you want to print something more).

# 5 Hint

Consider example 5 with directory list:

```
usr/bin/bash
etc/slurm
var/log
;
```

and the following output:

```
.
├── etc
│   └── slurm
├── usr
│   └── bin
│       └── bash
└── var
    └── log
```

Let's focus on a few select lines for hinting. For example when printing the line for `bash`. Notice:

1. `bash` is a subdirectory of `usr`. `usr` isn't the last item of its parent.
2. `bash` is also a subdirectory of `bin`. `bin` is the last item of its parent.
3. `bash` is the last item of its parent.

Based on that sequencing, we will therefore print out:

1. `│`
2. 
3. `└──`
4. `bash`

Another example is `slurm`. Notice:

1. `slurm` is a subdirectory of `etc`. `etc` isn't the last item of its parent.
2. `slurm` is the last item of its parent.

Based on that sequencing, we will therefore print out:

1. `│`
2. `└──`
3. `slurm`

Hopefully these examples show you the pattern to figure it out.